HashiCorp Vault in the Media Insight Engine

Complete reference: setup, secrets, PKI, and how everything connects

This document explains, end to end:

- what Vault stores and why nothing sensitive lives in the repo;
- how Vault is configured in all three deployment paths (Docker Compose, the Helm chart, and the Kustomize/minikube stack);
- exactly how the backend obtains its database credentials and API key;
- how the database uses Vault-issued certificates;
- how the frontend connects, and why it never touches Vault;
- the local (dev-mode) reality versus a production-hardened Vault.

Every claim points at the file it came from, so you can open the source next to this page and follow along. Line numbers are approximate where the source may drift.

Part 0 - What Vault is doing here

In one sentence

Vault is the single source of every secret the system needs at runtime - the Postgres credential, the Anthropic API key, and the TLS private keys - so that none of them is ever committed to the repo, baked into an image, or written to disk. Workloads ask Vault for what they need when they start, and Vault hands it over only after proving who is asking.

The three things Vault provides

- **KV secrets** the app's .env values (ANTHROPIC_API_KEY, etc.) and the Postgres password, stored as key/value secrets.
- **PKI engine** a private certificate authority that mints short-lived TLS certificates on demand for the mutual-TLS login between the server and Postgres.
- **Authentication** a way for a workload to prove its identity (a Kubernetes service account, or a pre-issued token) and be mapped to a least-privilege policy.

The one idea behind the whole design

The database login is a certificate, not a password. Postgres is configured to accept a client that presents a certificate signed by a trusted CA whose Common Name (CN=mie) equals the database role. Vault owns that CA and issues the certificate. So 'connecting to the database' and 'getting a certificate from Vault' are the same act - and there is no DB password anywhere in the application path.

Three deployment paths, one concept

The project can be run three ways. All three use Vault for the same purpose, but they differ in how Vault runs and how the certificate reaches the workload. The table below is the map; Parts 2-4 then walk each path in detail.

How the three paths compare

```
Aspect       | Compose          | Helm charts/mie  | Kustomize k8s/
-------------+------------------+------------------+--------------------
Target       | laptop (docker)  | kind cluster     | minikube (prod-ish)
Vault runs   | 1 container, dev | 1 Deployment,dev | official chart, dev
             |                  |                  | + Agent Injector
Cert deliver | KV + 1 token     | KV + 2 tokens    | PKI issued on the
             | (app reads it)   | (app + db read)  | fly, via a sidecar
App -> Vault | yes (secrets.py) | yes (secrets.py) | no (agent does it)
DB_AUTH      | cert             | cert             | password*
Vault login  | root-made token  | root-made tokens | Kubernetes SA auth

* password here means 'use DATABASE_URL as-is' - the URL already points
  libpq at the certificate files the Vault agent wrote. Still mutual TLS.
```

Part 1 - How Vault itself is set up

Dev mode, everywhere (and why)

In all three paths Vault runs in DEV MODE: a single process, storage held in memory, automatically unsealed, reachable with a fixed root token ('root'). This is deliberate and local-only. It makes the stack start with one command and removes the operational burden of unseal keys while developing.

docker-compose.yml:3-14 / charts/mie values + k8s/charts/vault/values.yaml

```
vault:
  image: hashicorp/vault:1.18
  command: server -dev            # in-memory, auto-unsealed. Local only.
  environment:
    VAULT_DEV_ROOT_TOKEN_ID: root
    VAULT_DEV_LISTEN_ADDRESS: 0.0.0.0:8200
```

The consequence of in-memory storage: when the Vault pod/container restarts it FORGETS everything - all KV data, PKI, auth config. Every path therefore re-seeds Vault on each deploy. That re-seed step is idempotent, so running it repeatedly is safe.

What gets stored in Vault

- **KV v2** the KV v2 secrets engine, mounted at secret/. Holds the app's environment and the Postgres password (and, in the token-based paths, the certificate bundles).
- **PKI** the PKI secrets engine (Kustomize path only), mounted at pki/. Holds a root CA and two certificate 'roles' that constrain what names a cert may carry.
- **Auth** one or more auth methods: a root-minted token (Compose and Helm), or the Kubernetes auth method that verifies a pod's service-account token (Kustomize).

Least privilege is enforced by policy

Whatever the login method, each workload is bound to a Vault POLICY that grants the minimum. The server may read only its own secret and (in the PKI path) issue only its own client certificate; Postgres may read only the DB password and issue only its own server certificate. Neither can read the other's secret or mint the other's cert. Concretely:

k8s/charts/vault/policies/server.hcl (Kustomize path)

```
path "secret/data/mie/__ENV__/app"      { capabilities = ["read"] }
path "pki/issue/mie-__ENV__-db-client"  { capabilities = ["update"] }
path "pki/cert/ca"                       { capabilities = ["read"] }
```

k8s/charts/vault/policies/postgres.hcl (Kustomize path)

```
path "secret/data/mie/__ENV__/postgres" { capabilities = ["read"] }
path "pki/issue/mie-__ENV__-db-server"  { capabilities = ["update"] }
path "pki/cert/ca"                       { capabilities = ["read"] }
```

In the token-based paths the same split is expressed as two one-path policies: 'mie-db' reads secret/data/mie/db-client, 'mie-pg' reads secret/data/mie/db-server.

Part 2 - Path A: Docker Compose (token + KV)

In one sentence

A one-shot 'cert-init' container generates the CA and certificates, stores the client bundle in Vault's KV, writes a read-only scoped token to a shared volume, and places the server certs where Postgres can read them. At startup the server reads the client bundle from Vault using that token and connects with mutual TLS.

1. **cert-init mints the certificates and seeds Vault**
   The cert-init service (the Vault image, run as root) runs openssl to create a CA, a server cert whose SAN is 'db', and a client cert whose CN is 'mie'. The client bundle is written to Vault KV; a least-privilege policy and a 168h token scoped to just that path are created for the app.

scripts/cert-init.sh (seed Vault + scoped token)

```
vault kv put secret/mie/db-client \
  ca.crt=@ca.crt client.crt=@client.crt client.key=@client.key
vault policy write mie-db - <<EOF
path "secret/data/mie/db-client" { capabilities = ["read"] }
EOF
vault token create -policy=mie-db -ttl=168h -field=token \
  > /certs/shared/vault-token
```

2. **docker-compose wires the four services together**
   Vault (dev), cert-init (one-shot), db (Postgres with ssl=on), and server. Shared volumes carry the server certs to Postgres and the scoped token to the server. depends_on + healthchecks order the startup: Vault healthy -> cert-init completes -> db healthy -> server.

docker-compose.yml:61-85 (server service)

```
environment:
  DB_AUTH: cert                # mutual TLS; cert fetched from Vault
  DATABASE_URL: postgresql://mie@db:5432/mie   # no password
  VAULT_ADDR: http://vault:8200
  VAULT_TOKEN_FILE: /certs-shared/vault-token
  VAULT_DB_CERT_PATH: mie/db-client
  DB_CERT_DIR: /run/mie-certs
tmpfs:
  - /run/mie-certs             # client key/cert live in RAM only
```

3. **The server reads the bundle from Vault at startup**
   The application code (Part 5) reads the token file, GETs the KV secret over HTTP, writes the three files 0600 into the tmpfs dir, and connects with sslmode=verify-full. The private key never touches disk - tmpfs is RAM. See Part 5 for the exact code, which is shared with Path B.

Part 3 - Path B: Helm chart (fixed-id tokens + KV)

In one sentence

The same token-and-KV idea as Compose, adapted to Kubernetes. A cert-init Job seeds BOTH the client and the server bundle into Vault KV and mints two FIXED-ID tokens. The server reads the client bundle with one token; the database's own init-container pulls the server bundle with the other - 'Vault as the source' for both sides.

1. **A cert-init Job seeds both bundles and mints two tokens**
   The Job waits for the dev Vault to be unsealed, then runs cert-init-k8s.sh: generate CA + server cert (SAN=db,localhost) + client cert (CN=mie); put both bundles in KV; write two one-path policies; mint two tokens with fixed ids (revoking first so re-runs don't collide).

charts/mie/files/cert-init-k8s.sh (seed + policies + tokens)

```
vault kv put secret/mie/db-client  ca.crt=@.. client.crt=@.. client.key=@..
vault kv put secret/mie/db-server  ca.crt=@.. server.crt=@.. server.key=@..
vault policy write mie-db ...   # read secret/data/mie/db-client
vault policy write mie-pg ...   # read secret/data/mie/db-server
vault token create -id=$SERVER_TOKEN -policy=mie-db -ttl=168h ...
vault token create -id=$DB_TOKEN     -policy=mie-pg -ttl=168h ...
```

2. **The token ids live in one Kubernetes Secret**
   The chart renders a Secret holding the API key, the Postgres password, and the two fixed token ids. Every workload reads the tokens from this one place. Note the intentionally counter-intuitive naming: the SERVER token has policy mie-db (reads db-client); the DB token has policy mie-pg (reads db-server).

charts/mie/templates/secret.yaml + values.yaml

```
ANTHROPIC_API_KEY:  <required at install>
POSTGRES_PASSWORD:  <required at install>
VAULT_SERVER_TOKEN: mie-server-tok   # policy mie-db -> db-client (app)
VAULT_DB_TOKEN:     mie-db-tok       # policy mie-pg -> db-server (db)
```

3. **The server deployment reads the client bundle**
   DB_AUTH=cert and VAULT_TOKEN comes from VAULT_SERVER_TOKEN. Two init-containers gate startup: 'wait-vault' polls until the db-client bundle exists in KV; 'wait-db' polls pg_isready. The client key/cert are written to an in-memory emptyDir (tmpfs parity with Compose).

charts/mie/templates/server-deployment.yaml (env)

```
DB_AUTH: cert
DATABASE_URL: postgresql://mie@db:5432/mie
VAULT_ADDR: http://vault:8200
VAULT_TOKEN: <- secret key VAULT_SERVER_TOKEN
VAULT_DB_CERT_PATH: mie/db-client
DB_CERT_DIR: /run/mie-certs   # emptyDir medium: Memory
```

4. **The database init-container pulls its server bundle**
   Before Postgres starts, a 'cert-pull' init-container (run as root) logs in with the DB token, reads the db-server bundle from KV, writes server.crt/key + ca.crt to a shared dir, and chowns the key to uid 999 so Postgres accepts it. Postgres then starts with ssl=on reading those files.

charts/mie/templates/db-statefulset.yaml (cert-pull init)

```
VAULT_TOKEN: <- secret key VAULT_DB_TOKEN
vault kv get -field=server.crt secret/mie/db-server > /certs/server.crt
vault kv get -field=server.key secret/mie/db-server > /certs/server.key
vault kv get -field=ca.crt     secret/mie/db-server > /certs/ca.crt
chown 999:999 /certs/server.key && chmod 600 /certs/server.key
```

Part 4 - Path C: Kustomize / minikube (Agent Injector + PKI)

In one sentence

The most production-shaped path. Vault runs from the official Helm chart with the Agent Injector enabled. Pods authenticate with their Kubernetes service account - no tokens are pre-shared. An injected sidecar logs in, asks the PKI engine for a FRESH short-lived certificate, and renders it into an in-memory file the app and Postgres read.

1. **Install Vault with the Agent Injector**
   minikube-up.sh installs hashicorp/vault chart 0.34.1 into the 'vault' namespace in dev mode, with the Agent Injector (a mutating admission webhook) and the UI enabled.

k8s/scripts/minikube-up.sh / k8s/charts/vault/values.yaml

```
helm upgrade --install vault hashicorp/vault --version 0.34.1 \
  --namespace vault --create-namespace -f charts/vault/values.yaml
# values: server.dev.enabled=true, injector.enabled=true, ui.enabled=true
```

2. **vault-bootstrap.sh configures KV, PKI, and Kubernetes auth**
   Run per environment (dev|test|prod) on every deploy. It loads .env into KV, enables the PKI engine with a root CA and two cert roles, and enables Kubernetes auth with a role per service account bound to its policy.

k8s/scripts/vault-bootstrap.sh (the three sections)

```
# KV:  secret/mie/<env>/app       (every .env key except the DB password)
#      secret/mie/<env>/postgres  (POSTGRES_PASSWORD, kept apart)
# PKI: root CA 'mie-local-ca'; roles mie-<env>-db-server (max 720h)
#      and mie-<env>-db-client (CN=mie, 24h / max 72h)
# AUTH: kubernetes; role mie-<env>-<who> binds SA mie-<who> in mie-<env>
#       to policy mie-<env>-<who>, checking the token audience
```

3. **A pod opts in with annotations; the injector adds a sidecar**
   Any pod annotated agent-inject:true is rewritten at admission to gain a vault-agent-init container and a vault-agent sidecar. The agent logs in via Kubernetes auth with the pod's service-account token (Vault verifies it with the API server, audience included), maps it to the role + policy, and renders templates into /vault/secrets (an in-memory volume).

k8s/overlays/dev/vault-patch.yaml (server pod annotations)

```
vault.hashicorp.com/role: mie-dev-server
# app.env: every KV key as an 'export K=V' line, sourced before uvicorn
agent-inject-secret-app.env:     secret/data/mie/dev/app
# db-client.pem: cert+CA+key from ONE pki issue call (cert and key match)
agent-inject-secret-db-client.pem: pki/issue/mie-dev-db-client
#   template: issue common_name=mie ttl=24h  -> cert, issuing_ca, key
agent-inject-secret-ca.crt:      pki/cert/ca
```

Postgres is injected the same way with role mie-dev-postgres: a server.pem issued for CN=postgres (+ the in-cluster DNS names) at ttl=720h, the CA, and pgpass read from KV. It runs the agent as the same uid (999) so Postgres accepts the key's ownership.

4. **The server does NOT talk to Vault here**
   Because the sidecar already wrote the certificate files, the app runs with DB_AUTH=password, which means 'use DATABASE_URL exactly as given'. The URL simply points libpq at the agent's files. The login is still mutual TLS - the app just doesn't fetch anything itself.

k8s/manifests/configmaps/server-env.yaml

```
DATABASE_URL: postgresql://mie@postgres:5432/mie
  ?sslmode=verify-full
  &sslrootcert=/vault/secrets/ca.crt
  &sslcert=/vault/secrets/db-client.pem
  &sslkey=/vault/secrets/db-client.pem
DB_AUTH: password
```

Part 5 - How the backend connects (the app code)

One function, swappable by design

The rest of the app never knows where certificates come from. db.py asks for SSL parameters; secrets.py produces them. In the token paths (A and B) secrets.py reads Vault; in the PKI path (C) secrets.py is not called at all because DB_AUTH is 'password'. The seam is one line.

server/app/db.py:17-28 (the decision, connect, pool)

```
ssl = secrets.ensure_db_certs() if settings.db_auth == "cert" else {}
with psycopg.connect(settings.database_url,
                     autocommit=True, **ssl) as conn:
    conn.execute(schema)          # under an advisory lock
_pool = ConnectionPool(settings.database_url, min_size=1, max_size=10,
    kwargs={"autocommit": True, "row_factory": dict_row, **ssl},
    configure=register_vector)
```

What ensure_db_certs() does

1. **Find the Vault token**
   Prefer VAULT_TOKEN from the environment; otherwise read VAULT_TOKEN_FILE. Fail fast with a clear message if neither is present.
2. **Read the KV v2 secret over HTTP**
   GET {VAULT_ADDR}/v1/secret/data/{path} with the X-Vault-Token header. KV v2 nests the values under .data.data in the JSON response. Missing keys or an unreachable Vault raise a clear RuntimeError so startup fails loudly rather than limping on.

server/app/secrets.py:29-43 (read path)

```
url = f"{vault_addr}/v1/secret/data/{vault_db_cert_path}"
resp = httpx.get(url, headers={'X-Vault-Token': _vault_token()})
data = (resp.json().get('data') or {}).get('data')   # KV v2 nesting
```

3. **Write the files 0600 to tmpfs and return libpq params**
   Create the cert dir 0700, write ca.crt/client.crt/client.key at 0600 (libpq refuses a world-readable key), map each filename to its libpq parameter, and return the dict with sslmode=verify-full.

server/app/secrets.py:16-17, 45-57

```
_CERT_FILES = {"ca.crt": "sslrootcert",
               "client.crt": "sslcert",
               "client.key": "sslkey"}
ssl = {"sslmode": "verify-full"}
for filename, param in _CERT_FILES.items():
    path = cert_dir / filename
    path.write_text(data[filename]); path.chmod(0o600)
    ssl[param] = str(path)
```

The settings that steer it

server/app/config.py:14-21 (env-driven, with defaults)

```
database_url  = DATABASE_URL  (postgresql://mie:mie@localhost:5437/mie)
db_auth       = DB_AUTH       (password | cert)
vault_addr    = VAULT_ADDR    (http://vault:8200)
vault_token / vault_token_file  (env wins; else read the file)
vault_db_cert_path = VAULT_DB_CERT_PATH  (mie/db-client)
db_cert_dir   = DB_CERT_DIR   (/run/mie-certs; tmpfs, RAM only)
```

Part 6 - How the database uses Vault

The certificate is the login

Postgres runs with TLS on (ssl=on) using a server certificate Vault issued, and a custom pg_hba.conf rulebook. The decisive rule says: for TLS connections to the mie database, authenticate by client certificate, and require that the certificate's CN equals the role name. Vault issues the client cert with CN=mie, so it maps straight onto the 'mie' role.

postgres/pg_hba.conf (the decisive rules)

```
# TYPE     DATABASE  USER  ADDRESS  METHOD
hostssl    mie       mie   all      cert clientcert=verify-full
host       mie       mie   all      scram-sha-256   # non-TLS fallback
```

verify-full on BOTH ends means each side proves its identity: Postgres checks the client cert against the CA and reads its CN; the app checks the server cert against the same CA and its hostname. No password crosses the wire on this path. Where the server cert comes from is the only thing that differs between paths:

- **Path A** Compose - cert-init places server.crt/key on a shared volume.
- **Path B** Helm - the db's cert-pull init-container fetches the bundle from Vault KV.
- **Path C** Kustomize - the Vault agent issues server.pem fresh from PKI and renews it.

Part 7 - How the frontend connects

The frontend never touches Vault

This is the important boundary. The client is a React app, built to static files and served by nginx. It speaks ONLY plain HTTP to the server's REST API. It has no Vault token, no database connection, and no certificates. Every secret and every TLS private key stays on the server side of that line. The browser's trust stops at the API.

client/nginx.conf (what the client container actually does)

```
location /api/ {
    proxy_pass http://server:8003/;   # strips /api, talks to the API
    proxy_buffering off;              # stream answers as they arrive
    proxy_read_timeout 300s;          # long Q&A / question-bank runs
}
location / {
    try_files $uri /index.html;       # SPA: client-side routing
}
```

Two wiring styles, same frontend

1. **Docker Compose - the browser hits the API directly**
   The client dev server is published on 127.0.0.1:5176 and is told the API base URL via VITE_API_URL=http://localhost:8003. The browser calls the server on port 8003 directly; nginx's /api proxy is used in the built image, not in the Vite dev server.

docker-compose.yml:87-95 (client service)

```
client:
  environment:
    VITE_API_URL: http://localhost:8003
  ports: ["127.0.0.1:5176:5176"]
  depends_on: [server]
```

2. **Kubernetes - the Ingress routes everything through the client**
   An Ingress sends the host (mie-<env>.local) to the client Service. The client's nginx serves the app and proxies /api/* to the server Service (named 'server', port 8003). The browser only ever talks to one origin; nginx forwards API calls internally.

k8s/manifests/ingresses/mie.yaml + deployments/client.yaml

```
Ingress host mie.local  ->  Service client (port http)
client nginx  /          ->  static React app
client nginx  /api/*     ->  Service server : 8003
```

The full chain, front to back

A browser request that ends in a database query

```
browser
  -> client nginx            (serves app; proxies /api/*)
    -> server (FastAPI, 8003)
      -> uses certs from Vault (fetched by app, or by the agent)
        -> Postgres over mutual TLS (client cert CN=mie)
          -> rows back up the same chain
```

Part 8 - Local (dev mode) vs production

What is dev-grade today

The certificate-based DB login is real mutual TLS in every path. What is dev-grade is the VAULT itself and the DELIVERY around it: in-memory storage, auto-unseal, a fixed root token, and re-seeding on every restart. That is correct for a laptop or a throwaway cluster, and wrong for anything holding real secrets.

The production-hardened shape

Dev mode here vs a production Vault

```
Concern     | Here (dev mode)      | Production
------------+----------------------+---------------------------
Storage     | in memory, 1 pod     | Raft (integrated), 3-5 pods
Unseal      | auto, root token     | cloud KMS auto-unseal;
            |                      | recovery keys split offline
Root token  | used for everything  | revoked after bootstrap;
            |                      | OIDC/SSO for humans
Listener    | http://              | TLS end to end
Root CA     | generated in pki/    | offline root + intermediate
Secrets in  | seeded from .env     | Terraform for config; values
            |                      | never in git; dynamic DB creds
Audit       | none                 | file+syslog audit -> SIEM
Backups     | none (ephemeral)     | automated Raft snapshots,
            |                      | restore tested
```

A pragmatic shortcut for a small team: keep this exact app-side design (Kubernetes auth, the Agent Injector, PKI, least-privilege policies) and run the Vault control plane as a managed service - HCP Vault Dedicated or Vault Enterprise - which hands off HA, auto-unseal, patching, audit and backups. Self-host only with the appetite for the operations column above.

Where to look - file map

Vault configuration / seeding

- **scripts/cert-init.sh** Compose: generate certs, seed KV, mint the scoped token.
- **charts/mie/files/cert-init-k8s.sh** Helm: the same, adapted; both bundles + two fixed-id tokens.
- **k8s/scripts/vault-bootstrap.sh** Kustomize: KV + PKI (root CA, roles) + Kubernetes auth, per env.
- **k8s/scripts/minikube-up.sh** install the official Vault chart (dev + Agent Injector).
- **k8s/charts/vault/policies/*.hcl** least-privilege policies for the server and Postgres.
- **k8s/overlays/<env>/vault-patch.yaml** which role each pod uses and what the agent renders.

How workloads consume Vault

- **server/app/secrets.py** fetches the client bundle from Vault, returns libpq SSL params.
- **server/app/db.py** chooses cert vs password and opens the connection / pool.
- **server/app/config.py** the env-driven settings (VAULT_ADDR, token, paths, DB_AUTH).
- **k8s/manifests/configmaps/server-env.yaml** server env incl. the DATABASE_URL that points at the agent files.
- **charts/mie/templates/secret.yaml** the Kubernetes Secret holding the two fixed token ids.

Database and frontend

- **postgres/pg_hba.conf** the auth rule: hostssl ... cert clientcert=verify-full.
- **charts/mie/templates/db-statefulset.yaml** Postgres with ssl=on; init-container pulls its server cert.
- **client/nginx.conf** nginx serves the app and proxies /api/* to the server.
- **k8s/manifests/ingresses/mie.yaml** Ingress -> client; the client Service fronts everything.